Daily Log

This is where I'll record hopefully before class what we will be doing. Please check here and read before class, so you'll have an idea of what we will cover that day.

---

day 11 | 250922 M

---

day 10 | 250919 F

Integration is essentially addition and so there are fewer problems with dividing by a small number like we had with differentiation. But we still need to be careful about errors like round off. Also similar to differentiation, there is a forward and backward way to integrate, and neither of these are accurate enough to be useful, but there is something in between, in the case of integration that is the trapezoidal rule. And like with differentiation there will be a slightly more complicated method, but also even better than trapezoidal and that is Simpson's rule.

We will start with Riemann's definition of the integral:

$$\int_a^b f(x)\mathrm{d}x = \lim_{N\to\infty}\sum_{n=0}^{N-1} f(x_n)h = \lim_{N\to\infty}\sum_{n=1}^{N} f(x_n)h$$

This simply tells us that these right and left sums are the same when the divisions go to infinity. But of course we can't go to infinity. So, a trapezoidal rule is an average of both the right and left rectangle rules is a way of averaging both:

$$\int_a^b f(x)\mathrm{d}x \approx \sum_{n=0}^{N-1} \frac{f(x_{n+1}) + f(x_n)}{2}h$$

This has the effect after factoring out h/2 and expanding the sum of:

$$\int_a^b f(x)\mathrm{d}x \approx \frac{h}{2}[f(x_0) + 2f(x_1) + 2f(x_3) + \ldots + 2f(x_{N-1}) + f(x_N)]$$

Which can be rewritten as

$$\int_a^b f(x)\mathrm{d}x \approx \frac{h}{2}[f(a) + f(b)] + \sum_{n=1}^{N-1} f(x_n)h$$

And now we can calculate this sum much faster. Implementing this can have some "gotchas", so we have to be careful about how we go about pulling this calculation off.

Do exercise 5.1 to practice this. But also go back and work on exercise 3.8 as a call back to that chapter on graphing. This is a classic example of a problem that *seems* very difficult, but actually the author walks you through nicely.

---

day 9 | 250917 W

We used most of our time to talk about homework questions and clean up some previous comments. We also talked a good bit about calculus and specifically integration which we will do a lot of moving forward. We talked about Reimann sums and began to cook up a function that would do this for us, but ran out of time.

---

day 8 | 250915 M

Today we will practice plotting with real data that would be gathered from some instrument and which you want to plot for your scientific notebook or for a paper. Now that we have the basics of plotting we need to practice this with some real data and see what trouble we run into. So today we will import and trim data, and then prepare some plots based on that data.

To import some data to python, there are a variety of ways, each depending on how complex the data is. Today, we will start with a simple example of two columns of data in a file. The file extension is readable by python as simply a `comma separated value` or `csv` file. Many files that have various extensions are like this, and many/most experimental apparatus will have the ability to export data in such a format.

After loading the data using `np.loadtxt()`, we have to take the transform of this file using `data.T`, and then we are able to use matplotlib to plot the data just like before.

Pandas has a more complicated but more capable function called `pd.read_csv()`. This is our first encounter with data stored as a pandas `dataframe` object, but we will practice how to use it to plot some data.

---

day 7 | 250912 F

Now that we have the basics of plotting we need to practice this with some real data and see what trouble we run into. So today we will import and trim data, and then prepare some plots based on that data. We also want to check in and follow up on the last few assignments that you have, namely finishing printing out Pascal's triangle as well as finding a python graphing module and showing that off to the class.

To import some data to python, there are a variety of ways, each depending on how complex the data is. Today, we will start with a simple example of two columns of data in a file. The file extension is readable by python as simply a comma separated value or csv file. Many files that have various extensions are like this, and many/most experimental apparatus will have the ability to export data in such a format.

After loading the data using np.loadtxt(), we have to take the transform of this file using data.T, and then we are able to use matplotlib to plot the data just like before.

Next, we will choose a more complicated file, and talk through how we can import it using pandas. Pandas is a python library specifically for handling more complicated data files than numpy is good for. Notice for instance that pandas handles text headers to give a column a particular name. Each pandas column acts like an individual numpy array.

In this case we have a file that has a large header that we need to deal with, as well as a format problem. We will use a statement like this to load the data into what is called a "Data Frame", which is really just python's word for a database.

```
data = pd.read_csv('filename.txt', header=55, sep='\t',
encoding='windows-1252')
```

Now some of this is specific to this particular file, but these are some of the options you may need to use from time to time. Each column in this data file already has a name

This will lead us into a discussion of formats and what you can do with them. For example did you know that you can unzip a word document?

From here, work exercises 3.1 and 3.2.

---

day 6 | 250910 W

Pascal's Triangle

First we will work on Pascal's triangle, since I realized that you don't know about nested loops yet. So we'll get that to work and talk about those.

A nested loop always has a structure like this:

```
for i in some_kind_of_range:
    for j in some_other_range_or_list:
        do_some_stuff_with_i_and_j
    do_more_stuff_with_i
```

What this does is loop through one list and at each step loop through another list. These can be related to each other as in our example in class, or can be totally separate.

Chapter 3: plotting!

We will also begin working on plotting things. First we will just cook up some "data" by which I mean we will plot a particular function, and then we will look at some options around how to make this look nice. There are MANY options on how to plot things and change different settings, but we will stick to the simplest ones. Here is an example of how to plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0,10)
y = x**2

fig0 = plt.figure()
ax0 = fig0.add_subplot()

ax0.plot(x, y, 'o', mfc='none')
```

This will plot $y = x^2$ from 0 to 10. Notice how I put in the option `'0'` and `mfc='none'` because that is the kind of plot that *I* like.

---

day 5 | 250908 M

Now that we have talked through the major operators in python, we need to put this altogether with the python function. Functions in python are small packages of code that can take in input and execute a series of steps based on that input, and optionally return an output. Functions are defined beginning with def statements so something like this:

```
def myFunction(inputs, go, here, if, necessary):
    print('this is a function')
    output = inputs*go*here**(if+necessary)
    return(output)
```

Now, we can write functions that can perform a particular calculation with different inputs, and we can begin to keep some of these so we can refer to them again later.

We wrote a couple of functions in class, one for calculating factorials and another for understanding spherical coordinates. For homework do exercise 2.11 and take an honest crack at 2.12.

---

day 4

First here is a Markdown Cheat sheet: https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet

Today we need to get to our first new kind of operations in python, `if`, `while`, and `for`.

`if` is a conditional statement, that is if some kind of condition is met, then it will execute a block of code that is underneath it. If that condition is not met, then it won't. You can see how I use if/then statements in that last sentence, and that is basically how python works too. The `then` part of this command is what is executed inside of the `if` statement. IF that condition is not met, then a couple of things can be coded in to happening. Either nothing happens and the program continues with the next command that is issued, or you can put a `else` or `elif` commands and issue another set of instructions for the program to follow in the event that the `if` statement is not true. This true/false property gives us yet another data structure that we have not seen yet: the Boolean. It does not come up as a data type in its own right very often, but it is necessary to control `for` or `while` statements.

Similar to a `while` statement is a `for` loop. A `for` loop iterates over an input, and performs a function that is included within it. `for` and `while` loops can often be used interchangeably, but I find myself usually going to a `for` loop more often. The iterator can be many things, but we will start with base python `range` function and then also use numpy's functions `np.arange` and `np.linspace`. Much of what we do will be using `for` loops although there are some faster ways of doing things using a numpy array that will speed things up.

As an example, in class we wrote a `while` statement that printed the Fibonacci numbers up to 1000.

For homework, I want you to write a program that performs a sum of reciprocals. In mathematical language this would look like $s(k) = \sum_1^{100} \frac{1}{k}$. As a check on this, you should get about 5.187. I would use a `for` loop on this.

---

day 3

Today we will finish our discussion of data types, by talking about strings and talking about lists or arrays. We have already used strings to print statements in words to the terminal. They are an important class of data, but less used in physics simulations. However the most important data structure to us is the list, and the important upgrade to the array. We will learn to create and import a list, how to print it and will begin to see how to graph it.

A string is any kind of data stored between single quotes, such as `'Hello World'`. A string can contain numbers, but they are not stored or used as numbers but rather essentially as letters. If a number happens to be stored as a string and you would like to use it as a number, then you can use the `float` or `int` functions for this. There are a few other properties of strings that we can talk about once we get through the next section on *lists*.

A list is a collection of data structures, and while technically anything can go in the container, we will be dealing with lists mostly as a collection of float numbers.

---

day 2

Now that we have `conda` installed and working, we need to make sure it has all of the libraries that we will use. We will use matplotlib, numpy, pandas, and scipy for the most part although there may be others that would be nice. Let's try to use them first and then we will install them if necessary. To import these, we use an import statement like `import numpy as np`. This will bring all of the functions from the `numpy` package into our code, and then we can use them if we first use the `np` identifier. So for example we could now use `np.pi` to use numpy's value for pi. Keep in mind that this is slightly different from the way your book uses import statements. Our way might technically be slower overall, but we won't notice and this will allow us to explore things more easily.

We used a variable in Day 2, but a variable is simply a name or letter that stores a reference to another object in python. I have to *declare* a variable in order to use it, and in python that means I have to give it a value (even if it won't keep that value). `x` vs `x=5` vs `x=y`.

Also, python is very limited in the math that it can do for you. `x = 5*10` works just fine but `x=y/2` does not nor can python solve for another variable like `y`.

But something like `x = x + 1` works just fine (as long as x has been defined already), even though it makes no sense to us. In fact, we will do operations like this all

the time!

Let's do a few examples (and we'll cover comments along the way):

1. a ball dropped from a tower, given a height and time, tell where the ball is.
2. conversion from polar coordinates to cartesian

day 1

Quickly review what we talked about last time.

1. Powershell/Terminal, cd ~, ls, pwd, mkdir
2. Opening a jupyter lab instance in a directory where you want to store things

What we want to talk about today are the various ways that you can run a script with python. So we are going to look at

1. The interpretor
2. A python script
3. A jupyter lab/notebook session

We will mostly be using jupyter lab sessions as they are interactive and offer a lot of opportunity for experimentation.

We will also show along the way how variables work and how basic math operations work. Everything is mostly what you expect with the exception of exponents, where 2^5 would be written as 2**5 using python. We will also cover code comments along the way.

day 0

This is where I have to tell you what you should install. Technically, I hope that everyone could install the following software on their computer. These are listed in order of priority

1. Anaconda distribution of Python
2. Mathematica
3. Dropbox (create account, but also download and install)
4. Keepassxc (not essential but a good habit)
5. git (extra special sauce)

If you could begin an account with the following:

1. Dropbox (from above)

2. Overleaf (free, student edition)
3. Github (extra if you get git working)

Talk about AI. I want students to learn to use it as much as they learn to use the computer itself. But start off without it. If you do use it, but big, bright lines around what you got it to teach you.

I talked through the python versions of 1. PowerShell/Terminal 2. Script 3. Jupyter Notebook/Lab. We did a very simple 'hello world' as well as a for loop just to show the differences in each, and went over some typo level gotcha's that come up. End on Jupyter Lab and showed them how that is a nice mid point between the interpreter and a script.

For the interpreter, you should just use terminal on Mac/Linux, and use Powershell for Anaconda on Windows.

For the script, I talk through using a shebang like #!/usr/bin/env python. We looked at mistakes and I showed them how to use the Traceback to kind of find where the problem is, although this is not perfect.

For jupyter, we will but using lab not notebook but they are very similar and students should know how to use both.

Here is chapter 2 of the book: chapter 2

In [1]:
```
!jupyter nbconvert --output-dir='.' --output="README.md" --to markdown --Tem
```

```
[NbConvertApp] Converting notebook Daily_Log.ipynb to markdown
[NbConvertApp] Writing 11721 bytes to README.md
```

In [ ]:
```
```